

A Multimodal Deep Learning Framework for Integrated Breast Cancer Diagnosis and Molecular Subtyping

A Final Year Project Report

Department of Computer Engineering
New Horizon Institute of Technology and Management
University of Mumbai
Academic Year 2025-26

Abstract

Breast cancer comprises multiple molecular subtypes — Luminal A, Luminal B, HER2-enriched, and Triple-Negative — that differ substantially in prognosis and treatment strategy. Mammography-based classification of these subtypes remains difficult because two of the most clinically important subtypes, Luminal A and Luminal B, present with near-identical radiographic appearance; their distinction depends primarily on cellular proliferation markers not directly visible on an X-ray.

A 2025 study from Weill Cornell Medicine-Qatar demonstrated that a multimodal model combining mammography with basic clinical metadata could classify five breast cancer categories with an overall AUC of 88.87%, but its performance on Luminal A and Luminal B specifically was substantially weaker (67% and 74% AUC respectively), indicating that the imaging and metadata used carried little information relevant to that particular distinction.

This project proposes an extended architecture that addresses this specific weakness through four additions to the baseline approach: (1) dual-view mammography input (craniocaudal and mediolateral-oblique) instead of a single view, (2) lesion-localised cropping using the TOMPEI-CMMD segmentation annotations, published after the baseline study, (3) radiomic texture features computed from the true tumour boundary rather than the whole image, and (4) a gated fusion mechanism that learns how much to weight each modality on a per-patient basis. The system is designed as a two-stage pipeline — a screening stage for benign/malignant classification followed by a subtyping stage for malignant cases — intended for use as a radiologist decision-support tool rather than a standalone diagnostic or a replacement for biopsy-confirmed pathology.

This report documents the system architecture, the data processing pipeline, the experimental design used to isolate the contribution of each proposed component, and the deployment plan for a demonstration application.

1. Introduction

1.1 Clinical Motivation

Breast cancer is not a single disease but a group of molecularly distinct diseases that happen to arise in the same organ. Two subtypes in particular, Luminal A and Luminal B, are difficult to distinguish through medical imaging because their visual presentation on a mammogram is often similar. The distinction instead depends on the expression of a proliferation marker (Ki-67), which requires a tissue-level assay to measure directly. This distinction has direct treatment consequences: Luminal A patients typically respond well to hormone therapy alone, while Luminal B patients generally require chemotherapy in addition. A model that reliably narrows down the likely subtype from imaging and readily available clinical data could therefore support earlier, better-informed treatment planning.

1.2 Problem Statement

The baseline study used for comparison in this project (Section 2) achieved strong overall classification performance but performed close to chance level specifically on the Luminal A / Luminal B distinction. This project investigates whether that specific weakness can be addressed by giving the model more spatially and texturally precise information about the tumour itself — information the baseline architecture did not use — while keeping the comparison scientifically fair through a controlled ablation study (Section 6).

1.3 Scope and Terminology

This project targets molecular subtype classification (Luminal A, Luminal B, HER2-enriched, Triple-Negative) as a proxy for tumour aggressiveness and treatment planning. This is distinct from formal histopathological Nottingham grading (Grade 1/2/3), which requires a tissue biopsy and is not available in the dataset used here. This distinction is stated explicitly to avoid any ambiguity between the two related but non-equivalent classification schemes.

1.4 Intended Use

The system is designed as a decision-support second opinion for a radiologist or oncologist reviewing a mammography case, to be used alongside — not instead of — biopsy-confirmed diagnosis. It is not intended as a patient-facing application.

1.5 Hackathon Execution Scope (Built with Claude: Life Sciences, Jul 7–13, 2026)

Sections 1–9 describe the complete Final Year Project as originally scoped: a 10-week programme culminating in a full two-stage system with radiomics, gated fusion, and a deployed demonstration application. This section states explicitly which part of that plan is being executed and reported during the Built with Claude: Life Sciences hackathon (July 7–13, 2026), so that the scope of the hackathon submission is unambiguous and does not overstate what a single week can produce.

The hackathon submission covers Experiments E0 and E1 only (Table 6.1): reproducing the published baseline (single-view Xception, image-wise split) on the fixed TOMPEI-annotated subset, then replacing it with a patient-wise split and an EfficientNet-B3 backbone under otherwise identical conditions. The concrete, reproducible deliverable is a results table comparing the two AUC figures on the same cohort, the code that produced them, and an honest account of what changed and why — built using Claude Science for data exploration and experiment execution and Claude Code for pipeline engineering, and published as a public GitHub repository.

Lesion cropping (Section 3.3), radiomics extraction (Section 4.4), gated fusion, and the deployment pipeline (Section 7) remain part of the full project as described in Sections 1–9, but are explicitly out of scope for the seven-day hackathon window; they continue afterward as the remaining eight weeks of the Final Year Project timeline (Section 9).

2. Baseline Study

2.1 Reference Architecture and Reported Results

The architecture and results described in this section are drawn from the following published study, which serves as the direct point of comparison for this project:

Ben Rabah, C., Sattar, A., Ibrahim, A., Serag, A. “A Multimodal Deep Learning Model for the Classification of Breast Cancer Subtypes.” *Diagnostics* 2025, 15, 995. DOI: 10.3390/diagnostics15080995.

Aspect	Baseline Approach
Dataset	CMMD collection — 5,202 mammograms across 1,775 patients, filtered to a working set of 4,056 images
Image backbone	Xception, pretrained on ImageNet, fine-tuned end-to-end
Clinical features used	Patient age (binarized at 40) and lesion type (4-category encoding)
Fusion method	Direct concatenation of a 512-dimensional image embedding with a 32-dimensional clinical embedding (544-dim total)
Train / validation / test split	Image-wise 72% / 18% / 10% (not patient-wise)
Overall AUC (5-class)	88.87%
Luminal A / Luminal B AUC	67% and 74% respectively — the specific gap this project targets
Training hardware	NVIDIA A100 80GB, 10 epochs, batch size 64

Table 2.1 — Summary of the baseline architecture and reported results, independently verified against the published paper.

2.2 Availability of the Baseline Implementation

The published paper's Data Availability Statement indicates that source code would be released upon acceptance. As of this report, the associated repository ([github.com/serag-ai/Multimodal-BC-Classifier](https://github.com/serag-ai/Multimodal-BC-Classifer)) is publicly available and contains a README, a requirements file, and a set of training notebooks. This repository is used as a secondary cross-check during the reproduction of the baseline (Section 6), alongside the architecture description published in the paper itself, which remains the primary specification since the notebook implementation is not guaranteed to match every reported detail exactly.

2.3 Limitations of the Baseline Addressed in This Work

Limitation of the Baseline	Addressed By
No tumour localisation — model sees the whole breast	Lesion-cropped input using TOMPEI-CMMD segmentation masks (Section 3.3)
Single mammography view used	Dual-view input (CC + MLO) with a learned view-attention mechanism (Section 3.2, 3.3)
Only age and lesion type used as clinical context	Addition of radiomic texture features computed from the true lesion shape (Section 4.4)
Simple concatenation fusion	Gated fusion module that learns a per-patient trust weighting across modalities (Section 3.3)
Image-wise data split (risk of patient leakage between train/test)	Patient-wise stratified splitting (Section 5.1)

Table 2.2 — Baseline limitations and the corresponding design responses adopted in this project.

3. Proposed System Architecture

3.1 Annotation Coverage Constraint

The TOMPEI-CMMD annotation set, which provides the lesion segmentation masks used for localisation in this project, covers only the mediolateral-oblique (MLO) view. According to its dataset documentation on The Cancer Imaging Archive, the released clinical spreadsheet covers 2,436 breast images drawn from the full CMMD cohort of 1,775 patients, while the subset with an actual usable segmentation mask comprises 1,363 patients (1,385 lesion images). No craniocaudal (CC) view masks exist in this annotation set for any patient.

This constrains the architecture in one specific way: the lesion-cropped input used in Stage 2 (Section 3.3) can only be constructed from the MLO view. The CC view remains part of the model as a full, uncropped image in both stages, but it is never cropped to the lesion boundary, since no ground-truth mask exists for it.

3.2 Stage 1 — Screening (Benign vs. Malignant)

Component	Specification
Input A — CC view	Full mammogram, 224×224, uncropped
Input B — MLO view	Full mammogram, 224×224, uncropped (lesion cropping is Stage 2 only)
Input C — Clinical	Age (normalised) and lesion type (5-dimensional one-hot encoding)
Image encoder	EfficientNet-B3, pretrained, shared weights across the CC and MLO branches
View attention	Learned per-patient weighting between the CC and MLO feature vectors
Fusion	Concatenation of 512-dimensional image and 32-dimensional clinical embeddings (544-dim)
Output	Two classes: Benign / Malignant
Training cohort	Patients with a complete CC + MLO image pair (Section 5.3), drawn from the full CMMD cohort — TOMPEI mask availability is not a constraint at this stage

Table 3.1 — Stage 1 architecture specification.

3.3 Stage 2 — Molecular Subtype Classification

Component	Specification
Input A — CC view	Full mammogram, 224×224, uncropped
Input B — MLO view	Full mammogram, 224×224, uncropped, used by the image-encoder branch
Input C — MLO lesion crop	MLO image cropped to the TOMPEI-CMMD polygon mask bounding box (MLO only, per Section 3.1)
Input D — Radiomics	Twenty texture features extracted from Input C using the true polygon-derived mask (Section 4.4)
Input E — Clinical	Age and lesion type (5-dimensional one-hot encoding)
Image encoder	EfficientNet-B3 with view attention (CC + MLO), trained independently from Stage 1
Radiomics branch	Dense(20→64) → ReLU → Dropout(0.2)

Component	Specification
Clinical branch	Dense(5→32) → ReLU
Fusion	Gated fusion combining 512 (image) + 64 (radiomics) + 32 (clinical) = 608-dim, with a learned trust gate
Output	Four classes: Luminal A / Luminal B / HER2 / Triple-Negative
Training cohort	Patients with both a confirmed subtype label and a TOMPEI-CMMD mask (Section 5.2)

Table 3.2 — Stage 2 architecture specification.

4. Data Processing Pipeline

4.1 Preprocessing Framework

The imaging pipeline is implemented using MONAI, a PyTorch-based framework for medical imaging. It is used in preference to a hand-written pydicom/PIL pipeline for three reasons: (1) DICOM VOI LUT windowing is applied automatically and correctly, removing a common source of manual preprocessing error; (2) image and mask are loaded and transformed together as a paired dictionary, so any spatial transform applied to the image is guaranteed to apply identically to its mask; and (3) MONAI's CacheDataset stores preprocessed results after the first pass, allowing iterative development on modest hardware.

```
pip install monai pydicom SimpleITK
```

4.2 Mask Generation from Segmentation Annotations

The TOMPEI-CMMD annotations are provided as JSON polygon coordinates rather than raster masks. These are converted to binary NIfTI mask images so that MONAI can load each mask alongside its corresponding DICOM image as a matched pair.

```
import json, numpy as np, SimpleITK as sitk
from PIL import Image, ImageDraw

def tompei_json_to_mask(json_path, image_width, image_height):
    with open(json_path) as f:
        data = json.load(f)
        mask = Image.new('L', (image_width, image_height), 0)
        draw = ImageDraw.Draw(mask)
        for annotation in data:
            points = [(pt['x'], pt['y']) for pt in annotation['cgPoints']]
            if len(points) >= 3:
                draw.polygon(points, fill=255)
        return np.array(mask)

mask_array = tompei_json_to_mask(json_path, img_w, img_h)
mask_img = sitk.GetImageFromArray((mask_array > 0).astype('uint8'))
sitk.WriteImage(mask_img, f'masks/{patient_id}_mlo_mask.nii.gz')
```

4.3 Transform Pipelines

Two separate MONAI transform pipelines are used: one for the full, uncropped mammogram views used throughout both stages, and one specifically for the MLO lesion crop used in Stage 2.

```
from monai.transforms import (
    LoadImaged, EnsureChannelFirstd, ScaleIntensityd,
    Resized, CropForegroundd, RandFlipd, ToTensord, Compose
)

# Pipeline A -- full mammogram, CC and MLO, no mask
full_view_transforms = Compose([
    LoadImaged(keys=['cc_image', 'mlo_image']),
    EnsureChannelFirstd(keys=['cc_image', 'mlo_image']),
    Resized(keys=['cc_image', 'mlo_image'], spatial_size=(224, 224)),
    ScaleIntensityd(keys=['cc_image', 'mlo_image']),
    RandFlipd(keys=['cc_image', 'mlo_image'], prob=0.5, spatial_axis=1),
    ToTensord(keys=['cc_image', 'mlo_image']),
])

# Pipeline B -- MLO lesion crop only, uses the true polygon mask (Stage 2)
lesion_transforms = Compose([
    LoadImaged(keys=['mlo_image', 'lesion_mask']),
```



```

    EnsureChannelFirstd(keys=['mlo_image', 'lesion_mask']),
    CropForegroundd(keys=['mlo_image', 'lesion_mask'],
                    source_key='lesion_mask', margin=20),
    Resized(keys=['mlo_image', 'lesion_mask'], spatial_size=(224, 224)),
    ScaleIntensityd(keys=['mlo_image']),
    ToTensord(keys=['mlo_image', 'lesion_mask']),
1)

```

CropForegroundd crops both the image and its mask identically to the mask's bounding box (with a 20-pixel margin), which is what allows the radiomics extraction step (Section 4.4) to operate on a spatially correct crop rather than an approximate one.

4.4 Radiomic Feature Extraction

Radiomic features are extracted using PyRadiomics, applied to the true polygon-derived lesion mask produced in Section 4.2 — not an approximate rectangular region. This distinction is significant for shape-based features specifically: computing Shape2D features (perimeter, sphericity, elongation) against a rectangular approximation of the tumour boundary would only describe the crop rectangle's own geometry rather than the tumour's actual shape, which would defeat the purpose of including shape features in the first place. Using the true mask ensures the shape-derived features carry genuine tumour-morphology information, which is relevant to the Luminal A / Luminal B distinction targeted by this project.

Note on array shape: MONAI's EnsureChannelFirstd transform (Section 4.3) produces channel-first arrays of shape (1, H, W). Converting such an array directly with SimpleITK's GetImageFromArray produces a 3-dimensional image (with a depth of 1), which causes PyRadiomics' Shape2D extractor to fail, since it requires a genuinely 2-dimensional input. The channel dimension must be squeezed out before conversion, as shown below.

```

import numpy as np
import SimpleITK as sitk
from radiomics import featureextractor

extractor = featureextractor.RadiomicsFeatureExtractor(
    binWidth=25, normalize=True, normalizeScale=100)
extractor.enableFeatureClassByName('glcm')
extractor.enableFeatureClassByName('glrlm')
extractor.enableFeatureClassByName('shape2d')
extractor.enableFeatureClassByName('firstorder')

# cropped_image / cropped_mask are the channel-first (1, H, W) outputs of
# lesion_transforms. Squeeze the channel dimension so SimpleITK treats
# them as 2D images -- required for Shape2D to run at all.
image_2d = np.asarray(cropped_image).squeeze(0)
mask_2d = np.asarray(cropped_mask).squeeze(0).astype('uint8')

image_sitk = sitk.GetImageFromArray(image_2d)
mask_sitk = sitk.GetImageFromArray(mask_2d)

features = extractor.execute(image_sitk, mask_sitk, label=1)
rad_values = {k: float(v) for k, v in features.items()
              if k.startswith(('original_glcm', 'original_glrlm',
                              'original_shape2d', 'original_firstorder'))}

```

4.5 Caching for Iterative Development

```

from monai.data import CacheDataset, DataLoader

train_ds = CacheDataset(data=train_manifest,
                        transform=full_view_transforms, cache_rate=0.5)
train_loader = DataLoader(train_ds, batch_size=16, shuffle=True, num_workers=2)

```

cache_rate=0.5 caches half the dataset in memory after the first pass; this should be reduced to 0.2–0.3 on machines with limited RAM (8GB).

5. Data Preparation Protocol

5.1 Column Verification and Label Source

The CMMD clinical spreadsheet's classification column typically records only a benign/malignant label; the molecular subtype (Luminal A, Luminal B, HER2, Triple-Negative) is recorded in a separate column. Before any split logic is written, the real column names must be confirmed directly rather than assumed, since an incorrect assumption here would silently produce an unbalanced split without raising an error.

```
import pandas as pd

df = pd.read_excel('cmmd_clinical.xlsx')
print(df.columns.tolist())          # confirm exact column names before writing
further code
print(df['classification'].unique()) # confirm this is benign/malignant only
# identify the separate molecular-subtype column, e.g. 'subtype' or 'molecular_subtype'

patient_ids = df['ID1'].unique()      # verify 'ID1' is the correct patient-id column
patient_subtype = df.groupby('ID1')['ACTUAL_SUBTYPE_COLUMN_NAME'].first()
```

5.2 Determining the Usable Stage 2 Cohort

Since Stage 2 requires both a confirmed subtype label and a TOMPEI-CMMD mask, the exact size of this intersection must be computed directly from the downloaded data before any model code is written; this number is the basis for the fixed ablation cohort used throughout Section 6.

```
import os

subtype_patients = set(df[df['ACTUAL_SUBTYPE_COLUMN_NAME'].notna()]['ID1'].unique())
tompei_mask_files = os.listdir('tompei_json/')
tompei_patients = set(f.split('_')[0] for f in tompei_mask_files) # adjust to real
filenames
usable_patients = subtype_patients & tompei_patients

print(f'Patients with subtype label: {len(subtype_patients)}')
print(f'Patients with TOMPEI mask: {len(tompei_patients)}')
print(f'Usable for Stage 2 (E0-E6): {len(usable_patients)}')
```

5.3 View Pairing

Many patients have multiple images per breast side. Each patient-side combination must be paired to exactly one CC and one MLO image before Stage 1 can treat them as a matched pair of inputs.

```
def build_cc_mlo_pairs(df):
    pairs = []
    for (patient_id, side), group in df.groupby(['ID1', 'LeftRight']):
        cc_rows = group[group['view'] == 'CC']
        mlo_rows = group[group['view'] == 'MLO']
        if len(cc_rows) == 0 or len(mlo_rows) == 0:
            continue # incomplete pair -- exclude and log
        pairs.append({
            'patient_id': patient_id, 'side': side,
            'cc_path': cc_rows.iloc[0]['image_path'],
            'mlo_path': mlo_rows.iloc[0]['image_path'],
        })
    return pd.DataFrame(pairs)
```

The number of patients excluded for incomplete pairs should be logged and reported alongside the data flow, consistent with the exclusion reporting used in the baseline study.

5.4 Label Consistency

TOMPEI-CMMD provides corrected left/right and view labels relative to the original CMMD release. To avoid comparing experiments against a shifting ground truth, this project uses TOMPEI-CMMD's corrected labels consistently across every experiment, including the baseline reproduction. This is noted as a minor methodological difference from the original baseline study, since it means the reproduction runs on marginally cleaner label data than the original paper used. This correction applies specifically to the MLO images TOMPEI-CMMD annotates — it does not assess or correct CC images at all. CC-view labels are therefore retained as provided in the original CMMD release, and inherit whatever labelling accuracy that source has; this scope boundary is stated explicitly rather than implied.

6. Experimental Design

A central methodological requirement of this project is that the ablation study isolates the effect of each architectural addition rather than conflating it with a change in the underlying patient cohort. Since only a subset of patients have a usable TOMPEI-CMMD mask (Section 5.2), every Stage 2 experiment — including the baseline reproduction — is run on the same fixed cohort of patients who have both a confirmed subtype label and a mask. This is a deliberately smaller dataset than the full CMMD subtype-labelled cohort used in the original baseline study, and is reported as such.

Experiment	Change Introduced	Cohort Used
E0 — Baseline reproduction	Single-view Xception, age + lesion type, image-wise split, 224×224	Fixed TOMPEI-annotated subset
E1 — Revised baseline	EfficientNet-B3 backbone, patient-wise split, single view	Fixed TOMPEI-annotated subset
E2 — Dual-view	Adds MLO + CC with view attention (both uncropped)	Fixed TOMPEI-annotated subset
E3 — Lesion localisation	Adds MLO lesion crop via MONAI CropForegroundd	Fixed TOMPEI-annotated subset
E4 — Radiomics	Adds the radiomics branch computed from the true-mask crop	Fixed TOMPEI-annotated subset
E5 — Gated fusion	Replaces concatenation with the gated fusion module	Fixed TOMPEI-annotated subset
E6 — Full system	All components combined	Fixed TOMPEI-annotated subset

Table 6.1 — Ablation study design. E0 is re-run on the same fixed subset as E1–E6 rather than the full baseline dataset, so its AUC is reported separately from the originally published 88.87% baseline figure, which used the full dataset.

6.1 Combined Two-Stage AUC

Because the system is a two-stage pipeline, a defined procedure is needed to combine Stage 1 and Stage 2 outputs into a single comparable AUC figure against the baseline's single-stage 88.87%. For each test patient, Stage 1 is run first. If the predicted class is Benign, the final label is Benign; if Malignant, the same patient is passed to Stage 2, and its four-class output is scaled by the Stage 1 malignant probability to produce one calibrated five-class probability distribution (Benign, Luminal A, Luminal B, HER2, Triple-Negative) per patient. A one-vs-rest macro AUC is then computed across this combined distribution. This multiplicative combination is valid specifically because Stage 2 is trained exclusively on confirmed-malignant patients (Table 3.2), so its four-class output is already conditioned on malignancy; the combination would not be valid if Stage 2 had been trained on the full cohort including benign cases.

```
def get_chained_prediction(patient, stage1_model, stage2_model):
    stage1_probs = stage1_model.predict_proba(patient) # [P(benign), P(malignant)]
    if stage1_probs[0] > stage1_probs[1]:
        return {'Benign': stage1_probs[0], 'LumA': 0, 'LumB': 0, 'HER2': 0, 'TN': 0}
    else:
        stage2_probs = stage2_model.predict_proba(patient) # 4-class
        return {
            'Benign': stage1_probs[0],
            'LumA': stage1_probs[1] * stage2_probs[0],
            'LumB': stage1_probs[1] * stage2_probs[1],
            'HER2': stage1_probs[1] * stage2_probs[2],
```

```
    'TN': stage1_probs[1] * stage2_probs[3],  
}
```

7. Deployment

7.1 Hosting Platform

The demonstration application is deployed on Hugging Face Spaces rather than Streamlit Community Cloud. Free CPU Spaces on Hugging Face do go to sleep after a period of inactivity — currently a fixed 48-hour window — so this is not a platform that stays permanently warm on its own. The choice is still made for two concrete reasons: the 48-hour window is longer and more predictable than Streamlit Community Cloud's shorter, less transparently documented sleep behaviour, and direct control over the container build environment avoids build failures for compiled dependencies such as PyRadiomics that can occur under Streamlit Cloud's locked-down build process. An external uptime ping (Section 7.5) is used to keep the demo continuously available rather than relying on the 48-hour window alone.

7.2 Deployment Steps

- Create a free account at huggingface.co.
- Select “New Space,” choose a name, and select a license (MIT is appropriate for an academic project).
- For the SDK, select “Docker” and choose the Streamlit Docker template.
- Leave hardware at the default “CPU basic · 2 vCPU · 16GB · Free” tier — sufficient for inference with a pretrained EfficientNet-B3 backbone.
- Clone the Space repository locally: `git clone https://huggingface.co/spaces/<username>/<space-name>`
- Copy `app.py`, `requirements.txt`, trained model weights, and the `Dockerfile` into the cloned folder.
- Commit and push: `git add . && git commit -m "deploy" && git push` — the Space rebuilds automatically.

7.3 Containerization

```
FROM python:3.10-slim
RUN apt-get update && apt-get install -y \
    build-essential gcc g++ \
    && rm -rf /var/lib/apt/lists/*
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 7860
CMD ["streamlit", "run", "app.py", "--server.port=7860", "--server.address=0.0.0.0"]
```

Hugging Face Spaces expects the application to listen on port 7860 rather than Streamlit's default of 8501; the CMD instruction above accounts for this.

7.4 Dependency Management

```
streamlit==1.38.0
torch==2.2.0
torchvision==0.17.0
monai==1.3.2
pydicom==2.4.4
SimpleITK==2.3.1
pyradiomics==3.1.0
numpy==1.26.4
pillow==10.3.0
pandas==2.2.2
shap==0.45.1
```

```
grad-cam==1.5.2  
matplotlib==3.8.4
```

Pinning exact dependency versions, rather than package names alone, avoids version-mismatch build failures — SimpleITK and NumPy version conflicts are the most common cause of PyRadiomics failing to build.

7.5 Availability Monitoring

Since the free CPU tier sleeps after 48 hours of inactivity (Section 7.1), a free uptime-monitoring service (such as UptimeRobot or cron-job.org) should be configured to ping the Space's URL at an interval well under 48 hours. This is a required step for keeping the demo reliably available on demand, not an optional safeguard — without it, a Space left untouched over a weekend will be asleep the next time someone opens the link, incurring a delay of roughly 30–60 seconds while it restarts.

8. Limitations and Threats to Validity

This section states explicitly the boundaries within which the results of this project should be interpreted, so that the scope of the claims made is clear from the outset rather than surfacing only during evaluation.

8.1 Ceiling on Available Signal

The clinical distinction between Luminal A and Luminal B is defined by the Ki-67 proliferation index, a cellular-level marker measured through tissue biopsy rather than through any feature directly visible on a mammogram. The architectural additions in this project — lesion cropping, radiomics, gated fusion — are designed to extract the maximum information mammography can plausibly contain about tumour morphology, not to guarantee that this information is sufficient to fully resolve the Luminal A/B distinction. A modest improvement over the baseline's 67%/74% AUC would still be a meaningful, reportable finding about the limits of imaging-only subtype prediction, rather than a failure of the proposed architecture.

8.2 Cohort Size and Class Balance

The Stage 2 training cohort is restricted to patients with both a confirmed subtype label and a TOMPEI-CMMD mask (Section 5.2), which is smaller than the full CMMD subtype-labelled cohort and is not guaranteed to be evenly distributed across the four subtype classes. Class-weighted or focal loss should be used for Stage 2 training, and per-class sample counts should be reported alongside AUC rather than an aggregate macro AUC alone.

8.3 Error Propagation Between Stages

Since Stage 2 is only reached for patients Stage 1 classifies as malignant, any Stage 1 false negative permanently removes that patient from subtype prediction. Stage 1's false-negative rate on the malignant class should be reported explicitly alongside the combined AUC (Section 6.1), since it directly bounds the practical performance of the full pipeline regardless of Stage 2's own accuracy.

8.4 Statistical Validity of Ablation Comparisons

AUC differences between consecutive experiments in the ablation study (Section 6) should be accompanied by a bootstrap confidence interval or an equivalent significance test before being reported as evidence that a given architectural addition helped. On a cohort of this size, a small AUC difference between two experiments may not be statistically distinguishable from sampling noise.

8.5 External Validity

All experiments use data from a single collection (CMMD / TOMPEI-CMMD), one imaging protocol, and one patient population. No claim of generalisation to other scanners, institutions, or demographic groups is made or tested in this project; this is stated here as a limitation rather than left implicit.

8.6 Additional Design Trade-offs

Resizing all images to 224×224 is a standard choice for transfer-learning backbones but may discard fine detail — such as microcalcification texture — present in the original high-resolution mammogram; this is a deliberate trade-off for computational feasibility within the project timeline, not an oversight. Similarly, the radiomics branch (Section 4.4) may or may not add information beyond what the CNN backbone already extracts from the same lesion crop — this is precisely what the E3-to-E4 ablation step (Section 6) is designed to measure, and a null result at that step is a valid and useful outcome to report rather than a project failure.

9. Project Timeline

Week	Planned Work
1	TCIA account setup; download CMMD and TOMPEI-CMMD; verify real spreadsheet column names (5.1); compute the usable Stage 2 cohort size (5.2); build CC/MLO pairing (5.3).
2	Build the MONAI pipeline (Section 4): JSON-to-mask conversion, transform pipelines, CacheDataset; patient-wise splitting using the verified subtype column; finalise label source (5.4).
3	E0: reproduce the baseline architecture on the fixed TOMPEI-annotated subset, using the paper's architecture description as the primary specification and the now-available reference repository as a cross-check.
4	E1-E2: patient-wise split baseline and dual-view attention.
5	Radiomics extraction using the true mask (Section 4.4) on the fixed subset; feature selection (top 20 features).
6-7	E3-E6: lesion crop, radiomics branch, gated fusion, full system; complete the ablation table (Section 6).
8	Grad-CAM and SHAP interpretability analysis; generate figures, including a Luminal A vs. Luminal B comparison.
9	Compile the written report / paper draft.
10	Build and deploy the Streamlit demonstration application to Hugging Face Spaces (Section 7); record a demonstration video.

10. Key References and Resources

- CMMD dataset — cancerimagingarchive.net/collection/cmmd/
- TOMPEI-CMMD annotation set — cancerimagingarchive.net/analysis-result/tompei-cmmd/
- MONAI documentation — docs.monai.io
- PyRadiomics documentation — pyradiomics.readthedocs.io
- Hugging Face Spaces — huggingface.co/spaces
- Baseline study — Ben Rabah et al., Diagnostics 2025, 15, 995. doi.org/10.3390/diagnostics15080995
- Baseline reference implementation — github.com/serag-ai/Multimodal-BC-Classifer

Full install command:

```
pip install torch torchvision monai pydicom SimpleITK \
pillow numpy pandas scikit-learn tqdm \
pyradiomics shap grad-cam streamlit matplotlib openpyxl
```